

Aula 10 — Blazor WebAssembly (WASM)

Módulo: Blazor

Carga horária: 4 horas

Projeto base: Novo projeto Blazor WebAssembly + API ASP.NET Core

Objetivo da aula:

Compreender as diferenças entre **client-side** e **server-side**, analisar **performance** e **segurança** no Blazor WASM, habilitar **PWA** e implementar um **consumo real de API**, finalizando com **deploy local**.

PARTE 1 — TEORIA (≈ 2 horas)

1 Client-side vs Server-side

◆ Blazor Server

- Renderização no servidor
 - Comunicação via SignalR
 - Estado mantido no servidor
 - ✓ Menor download inicial
 - ✓ Acesso direto a EF Core
 - ✗ Depende de conexão contínua
-

◆ Blazor WebAssembly (WASM)

- Executa no navegador
- Código .NET compilado para WebAssembly
- Comunicação via HTTP (API)

- ✓ Escalável
- ✓ Funciona offline (PWA)
- ✓ Ideal para SPAs modernas
- ✗ Download inicial maior

📌 Comparativo Resumido

Aspecto	Server	WASM
Execução	Servidor	Navegador
Escala	Média	Alta
Offline	Não	Sim
Acesso DB	Direto	Apenas via API

2 Performance no Blazor WASM

Fatores críticos:

- **Tamanho do bundle**
- Lazy loading
- AOT (Ahead-of-Time)
- Caching

Boas práticas:

- Separar DTOs
- Minimizar bibliotecas
- Usar compressão (gzip/brotli)

3 Segurança

◆ Princípios

- Nunca acessar DB diretamente
- Toda regra sensível fica na API
- Validação dupla (client + server)

◆ Autenticação

- JWT (Bearer Token)
- OAuth2 / OpenID Connect
- Refresh Tokens

📌 Importante:

O código WASM é público — segurança **sempre** no backend.

4 PWA (Progressive Web App)

◆ Características

- Instalação no desktop/mobile
- Cache offline
- Atualização controlada

Blazor WASM suporta PWA nativamente:

```
--pwa
```

PARTE 2 — PRÁTICA (≈ 2 horas)

Objetivo Prático

Criar uma SPA em Blazor WASM, consumir uma API ASP.NET Core, habilitar PWA e realizar deploy local.

1 Criando a API ASP.NET Core

```
dotnet new webapi -n Aula10.Api cd Aula10.Api dotnet run
```

Controller da API

 Controllers/ProdutosController.cs

```
using Microsoft.AspNetCore.Mvc; namespace Aula10.Api.Controllers { [ApiController] [Route("api/[controller]")] public class
ProdutosController : ControllerBase { [HttpGet] public IActionResult Get() { var produtos = new[] { new { Id = 1, Nome =
"Notebook", Preco = 4500 }, new { Id = 2, Nome = "Mouse", Preco = 150 } }; return Ok(produtos); } } }
```

Executar:

```
https://localhost:5001/api/produtos
```

2 Criando o Projeto Blazor WebAssembly

```
dotnet new blazorwasm -n Aula10.BlazorWasm --pwa cd Aula10.BlazorWasm dotnet run
```

Estrutura:

3 Configurando HttpClient

 Program.cs

```
builder.Services.AddScoped(sp => new HttpClient { BaseAddress = new Uri("https://localhost:5001/") });
```

4 Criando o Model (DTO)

 Models/ProdutoDto.cs

```
namespace Aula10.BlazorWasm.Models { public class ProdutoDto { public int Id { get; set; } public string Nome { get; set; }  
public decimal Preco { get; set; } } }
```

5 Consumo da API no Blazor WASM

 Pages/Produtos.razor

```
@page "/produtos" @using Aula10.BlazorWasm.Models @inject HttpClient Http <h3>Produtos</h3> @if (produtos == null) {  
<p>Carregando...</p> } else { <ul> @foreach (var p in produtos) { <li>@p.Nome - @p.Preco.ToString("C")</li> } </ul> } @code {  
List<ProdutoDto>? produtos; protected override async Task OnInitializedAsync() { produtos = await  
Http.GetFromJsonAsync<List<ProdutoDto>>( "api/produtos"); } }
```

- ✓ Comunicação via HTTP
- ✓ DTO isolado
- ✓ Async/await

6 Habilitando PWA (verificação)

 wwwroot/manifest.json

```
{ "name": "Blazor WASM App", "short_name": "BlazorWASM", "start_url": "/", "display": "standalone" }
```

- ✓ Botão **Instalar** no navegador
- ✓ Cache offline automático

7 Deploy Local (Publish)

✓ Publicar

```
dotnet publish -c Release
```

Saída:

```
bin/Release/net8.0/publish/wwwroot
```

✓ Servir localmente

```
dotnet tool install --global dotnet-serve dotnet serve -d wwwroot
```

✓ Resultado Esperado

- ✓ SPA funcional em Blazor WASM
- ✓ Consumo real de API

- ✓ PWA habilitado
 - ✓ Deploy local funcionando
 - ✓ Base sólida para autenticação JWT
-

Atividade Avaliativa

Desafio prático:

1. Criar POST de Produto na API
2. Consumir POST no WASM
3. Exibir loading e erro
4. Implementar cache offline simples